RESEARCH

Evidence-Guided Crash Adjudication for LLM-assisted Fuzzing of Third-party Open-source Libraries in Connected Vehicles

Qingguan Gao[1*], Ying Zhang[1] and Letian Sha[2]

[1*]School of Cyber Science and Engineering, Southeast University, Nanjing, China.
[2]School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, China.

*Corresponding author(s). E-mail(s): gaoqingguan@seu.edu.cn;
Contributing authors: zhangying@seu.edu.cn; ltsha@njupt.edu.cn;

Abstract

Connected vehicle systems rely heavily on third-party open-source libraries, which necessitates effective fuzz testing to ensure robust security validation. Driven by the advancement of Large Language Models (LLMs), fuzz-driver generation has become increasingly automated. Nevertheless, directly applying LLMs to raw crash reports can produce many false alerts, while case-by-case triage incurs prohibitive labor costs. To tackle this problem, we propose CrashSage, an evidence-guided framework for post-fuzzing crash adjudication in LLM-assisted fuzzing. Before final LLM analysis, CrashSage prepares crash evidence through three key designs: rule-guided crash attribution, representative selection, and crash-context reconstruction. Rule-guided crash attribution filters low-value reports and likely driver-induced crashes to improve false-alert judgment. Representative selection compares exact and stack signatures among retained reports to identify repeated crash instances and submits only selected representatives to the LLM, reducing redundant analysis. Crash-context reconstruction enriches representative crashes with static and runtime evidence, helping determine whether the faulty state comes from the generated driver or the target component. Experiments on 1021 crash reports collected from 12 third-party open-source libraries show that CrashSage achieves 70.19% precision, 100.00% recall, and an 82.49% F1 score, while reducing LLM submissions from 1021 to 80 (92.16%). These results demonstrate its effectiveness in improving crash attribution and efficiency for LLM-assisted fuzzing of third-party open-source libraries in connected vehicles.

Keywords: fuzz testing, crash adjudication, large language model, crash attribution, connected vehicle security

1 Introduction

Intelligent connected vehicle (ICV) systems increasingly depend on third-party open-source libraries (Petit and Shladover 2015; Sun et al. 2022; Bernardini et al. 2017; Checkoway et al. 2011; Ladisa et al. 2023), causing ICV software to inherit security risks from these libraries. To expose crash-inducing defects in such libraries, fuzz testing is widely adopted (Böhme et al. 2019; Lemieux and Sen 2018; Ding and Le Goues 2021). Yet validating diverse third-party open-source libraries at scale still requires a large number of library-specific fuzz drivers. Recent LLM-assisted fuzzing systems attempt to ease this bottleneck by using LLMs to synthesize fuzz drivers automatically (Meng et al. 2024; Deng et al. 2024, 2023; Xia et al. 2024; Xu et al. 2025).

LLM-assisted fuzzing differs from conventional component fuzzing, where the harness is usually written or reviewed by developers and is therefore treated as trusted during crash inspection. In LLM-assisted fuzzing, the generated driver is not necessarily a trusted harness. It invokes target-component APIs with mutated inputs, but it may omit required initialization, construct invalid arguments, or introduce harness-level memory errors. As a result, a crash report no longer reflects target-component behavior alone, but the interaction between the generated driver and the target component.

This difference leads to three challenges in false-alert judgment, crash localization, and efficiency. First, existing crash adjudication tends to treat crash reports as direct evidence of target-component-attributed crashes, leaving driver-induced false alerts difficult to distinguish from target-component-originated crashes. Second, existing adjudication lacks localization evidence that combines static code context and runtime context, which can lead to inaccurate crash attribution. Third, repeated crash instances triggered during fuzzing may stem from the same underlying failure pattern, but existing workflows still analyze them separately, resulting in redundant triage, localization effort, and LLM submissions.

To address this problem, we propose CrashSage, an evidence-guided crash-adjudication pipeline tailored for LLM-assisted fuzzing with untrusted generated drivers. Rather than reasoning directly over raw crash reports, CrashSage applies rule-guided crash attribution to filter low-value reports and likely driver-induced failures, uses representative selection to reduce repeated crash instances, and performs crash-context reconstruction to localize the origin and propagation of the faulty state for the selected representatives. The resulting evidence is then submitted to LLM analysis for final crash diagnosis. In this way, the LLM reasons over filtered, representative, and localized crash evidence. CrashSage is designed as a post-fuzzing triage aid that reduces the number of reports requiring detailed inspection and prepares attribution evidence for retained representative crashes. The main contributions of this paper are listed as follows.

- **CrashSage pipeline.** We propose an evidence-guided crash-adjudication pipeline for LLM-assisted fuzzing. By preparing structured crash evidence before model reasoning, CrashSage reframes the LLM from a primary classifier over raw crash text into a final analyzer over curated execution evidence.
- **Rule-guided crash attribution.** We introduce rule-guided crash attribution over sanitizer signals and stack-frame roles. This stage filters low-value reports and likely driver-induced crashes before LLM analysis, mitigating over-attribution bias in prompt-based methods and improving false-alert judgment.
- **Representative selection.** We compare exact and stack signatures to identify repeated crash instances and retain representative crashes for subsequent LLM analysis. This selection process

reduces redundant analysis while preserving the relationship between each repeated crash and its representative.

- **Crash-context reconstruction.** We reconstruct static code context and runtime context for representative crashes to trace a crash across driver and component code. By combining source locations with execution-state evidence, this stage supports more accurate crash attribution and root-cause localization than static code context alone.
- **Experimental evaluation.** We evaluate CrashSage using 1021 crash reports collected from 12 third-party open-source libraries. Compared with the baseline, the full pipeline achieves 70.19% precision, 100.00% recall, and an 82.49% F1 score, while reducing end-to-end LLM submissions from 1021 to 80 (92.16%). Runtime context reconstruction and LLM analysis are the main sources of latency, while rule-guided crash attribution and representative selection each require less than 10 ms per report.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces the LLM-assisted fuzzing framework and its crash-adjudication challenges. Section 4 presents the design of CrashSage, Section 5 reports the evaluation, and Section 7 concludes the paper.

2 Related Work

2.1 LLM-Assisted Fuzzing

Fuzz testing remains a standard way to expose memory-safety defects in C/C++ software, especially with sanitizer support and continuous fuzzing infrastructure (Böhme et al. 2019; Lemieux and Sen 2018; Serebryany et al. 2012; Stepanov and Serebryany 2015; Ding and Le Goues 2021). In conventional component fuzzing, the driver is usually written or reviewed by developers, so downstream analysis can treat it as part of the trusted harness. However, writing such drivers for diverse libraries and APIs requires substantial manual effort, which limits the scalability of component-level fuzzing.

Recent LLM-assisted fuzzing systems address this scalability problem by using models to infer protocol knowledge, generate inputs, synthesize fuzz drivers, and repair compilation errors. ChatAFL (Meng et al. 2024) guides stateful protocol fuzzing with LLM-derived knowledge; FuzzGPT (Deng et al. 2024) and TitanFuzz (Deng et al. 2023) use LLMs for edge-case or test-input generation; Fuzz4All (Xia et al. 2024) generalizes LLM-based input generation across languages; PromptFuzz (Lyu et al. 2024) studies prompt strategies for C/C++ fuzz-driver generation; and CKGFuzzer (Xu et al. 2025) combines code-knowledge-graph construction with LLM-based driver generation and repair. Surveys of LLM-assisted testing likewise emphasize test generation, input synthesis, and repair (Wang et al. 2024). These systems lower driver-construction cost, but their main optimization target remains the upstream fuzzing workflow. Once a generated driver compiles and produces crashes, crash reports are usually treated as fuzzing outcomes rather than as evidence that must be adjudicated under an untrusted-harness assumption. As a result, existing LLM-assisted fuzzing work does not directly address whether a crash report reflects a driver-induced false alert or a target-component-originated crash.

2.2 Automated Crash Diagnosis and Reasoning

Given the post-fuzzing analysis gap left by LLM-assisted fuzzing, a natural question is whether existing crash diagnosis techniques can resolve the resulting crash reports. Early automated diagnosis methods relied on stack-trace signatures and heuristic rules for clustering, localization, and duplicate

detection. Igor (Jiang et al. 2021) clusters fuzz crashes by root cause; FuzzerAid (Joshy and Le 2022) groups crashes using learned fault signatures; CrashLocator (Wu et al. 2014) localizes crashing faults from crash stacks; FaST (Rodrigues et al. 2022) aligns stack traces efficiently; and ReBucket (Dang et al. 2012) clusters duplicate crash reports using call-stack similarity. Related methods enrich crash reports with source, execution, or explanation-oriented evidence: DeepAnalyze (Shetty et al. 2022) localizes crashing faults at scale, CrashTranslator (Huang et al. 2024) reconstructs executable scenarios from stack traces, and Better Debugging (Yan et al. 2024) combines static analysis with LLMs for explainable crash localization.

These techniques provide useful clustering and localization support, but they are most effective when crash evidence is stable and the harness is trusted. This assumption weakens under LLM-generated fuzz drivers. Analyses that do not explicitly localize the failure across driver and component code can mistake a component-visible crash site for a component-originated failure. Localization-oriented methods alleviate part of this ambiguity by recovering static code context, but existing execution enrichment is not designed to bind replay-consistent runtime context to driver/component attribution. Consequently, it may not reveal the runtime values, pointer states, or object states that determine whether the failure originates from driver-side construction or target-component behavior. Likewise, crash clustering can group repeated reports, but it does not by itself decide which representative should receive expensive LLM analysis or how duplicate reports should inherit the representative diagnosis.

LLM-based vulnerability reasoning appears to offer richer semantic judgment for this setting. CVE-LLM enriches vulnerability evaluation with cybersecurity ontologies (Ghosh et al. 2025), while AutoCVSS studies prompting, few-shot examples, and retrieval-augmented generation for CVSS prediction (Sanvito et al. 2025). However, these systems typically reason over curated vulnerability descriptions or structured security context, whereas LLM-assisted fuzzing often yields short crash reports from automatically generated drivers. Direct prompting over such reports lacks the structured semantics assumed by vulnerability-reasoning systems and can amplify over-attribution.

Overall, existing work leaves three gaps for LLM-assisted fuzzing crash adjudication. It does not reliably support false-alert judgment when sanitizer evidence mixes driver behavior with target-component behavior. It lacks attribution-oriented localization that combines static code context and runtime context, making it difficult to determine whether a failure propagates from the generated driver or originates within the target component. It also leaves efficiency concerns when repeated crash instances caused by the same underlying failure pattern are analyzed through separate LLM submissions. This motivates a pipeline that combines rule-guided crash attribution, representative selection, crash-context reconstruction, and LLM-based semantic reasoning before final crash diagnosis.

3 LLM-Assisted Fuzzing Framework and Challenges

3.1 LLM-Assisted Fuzzing Framework

Figure 1 illustrates the overall workflow of the LLM-assisted fuzzing framework considered in this work. The framework automates the process from modeling component APIs to generating fuzz drivers and collecting crash reports, and consists of three stages: API knowledge modeling, driver generation, and crash-oriented fuzzing (Xu et al. 2025).

API knowledge modeling. This stage first extracts the APIs to be tested from open-source components. Static feature extraction then collects key information about the target APIs and identifies their relationships. The extracted information is organized into an API knowledge graph, which

provides structured API knowledge for subsequent combination planning and driver generation (Xu et al. 2025).

Driver generation. The combination agent summarizes the target APIs and uses the API knowledge graph to plan API combinations. Each planned combination, together with the corresponding API source code and summaries, is provided to the driver generation agent, which synthesizes a fuzz driver that invokes the selected APIs (Meng et al. 2024; Xu et al. 2025; Lyu et al. 2024).

Crash-oriented fuzzing. For each generated driver, the seeds generation agent produces a set of initial seeds. The fuzzing engine iteratively mutates these seeds and executes the driver in an attempt to trigger crashes. When a crash occurs, the sanitizer records the failure and exports the corresponding crash report for subsequent crash adjudication (Böhme et al. 2019; Lemieux and Sen 2018; Serebryany et al. 2012; Stepanov and Serebryany 2015).

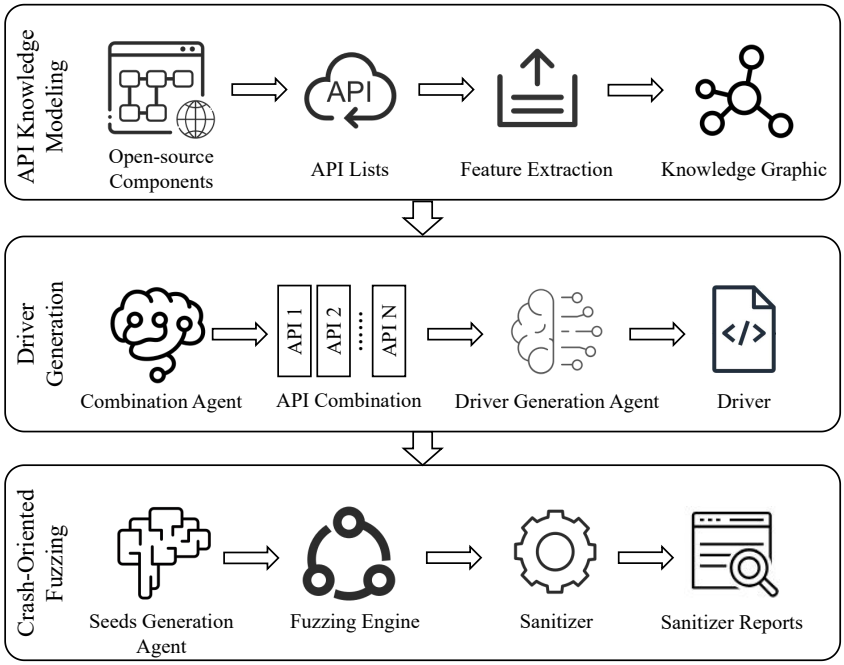


Fig. 1 Overview of the LLM-assisted fuzzing framework.

3.2 Challenges Statement

Unlike conventional fuzzing, LLM-assisted fuzzing cannot assume that the generated driver is a trusted harness (Xu et al. 2025; Lyu et al. 2024). A reported crash may result from the target component, the generated driver, or their interaction under a mutated input. Consequently, the crash report alone may not provide sufficient evidence for reliable post-fuzzing analysis. This setting raises three challenges: false-alert judgment, crash localization, and efficiency.

False-alert judgment. An LLM-generated driver may violate implicit API contracts by omitting required initialization, constructing inconsistent object states, or passing invalid argument combinations. Such misuse can trigger a failure while execution is inside the target component, causing the sanitizer stack trace to contain component frames even when the underlying fault originates from the driver. The presence of a component frame therefore does not by itself establish target-component attribution. Distinguishing driver-induced failures from target-component-originated crashes is difficult because a raw report records where the failure becomes observable, but does not explicitly identify which side violated the required execution contract.

Efficiency. LLM-assisted fuzzing repeatedly mutates related inputs, so many crashing inputs may exercise the same underlying failure pattern. Different generated drivers may also reach that failure pattern through distinct API combinations or call paths. These executions can produce reports that differ in addresses, stack depth, or intermediate frames despite sharing one root cause (Jiang et al. 2021; Joshy and Le 2022; Dang et al. 2012). Treating every report as an independent case leads to repeated inspection and redundant LLM submissions, making crash adjudication increasingly expensive as the number of generated drivers and crashing inputs grows.

Crash localization. Existing prompt-based analysis commonly reasons over raw crash reports without explicitly localizing the failure across the generated driver and the target component. Because a driver-constructed invalid state may become observable only after entering component code, the visible crash frame can be mistaken for evidence that the open-source supply-chain component caused the failure. Localization information is therefore necessary not only to identify a suspicious statement, but also to determine where the faulty state originates and how it reaches the crash site (Shetty et al. 2022; Huang et al. 2024; Yan et al. 2024; Wu et al. 2014). However, existing localization evidence is dominated by static code context, while runtime context is not systematically incorporated into driver/component attribution. Static code context cannot expose the concrete values, pointer states, or object states involved in the failing execution, leaving the propagation of the faulty state incomplete and crash attribution uncertain.

4 Approach

4.1 CrashSage Design Overview

In this section, we present the system design of CrashSage, an evidence-guided crash-adjudication framework for LLM-assisted fuzzing. CrashSage is designed to improve post-fuzzing crash adjudication when automatically generated fuzz drivers cannot be treated as trusted harnesses. Its primary objective is to prepare structured evidence that supports attribution-oriented localization, remove repeated crash instances, and retain auditable inputs before final LLM reasoning. Through this pipeline, CrashSage supports false-alert judgment, crash localization, and efficient analysis of repeatedly triggered crashes.

Figure 2 illustrates the workflow of CrashSage and aligns the three challenges with the corresponding processing stages. Initially, *rule-guided crash attribution* addresses false-alert judgment by distinguishing noise and likely driver-induced crashes from cases that require further target-component analysis. It normalizes crash reports and applies rule-based preliminary screening over sanitizer signals and stack-frame roles, producing filtered normalized crash reports for the following stages. Next, *representative selection* addresses efficiency by avoiding repeated analysis of crashes that follow the same underlying failure pattern. It constructs exact and stack signatures from the normalized sanitizer evidence, verifies signature matches, maps repeated crashes to an existing representative,

and retains unmatched crashes in the selected crash set. CrashSage then performs *crash-context reconstruction* to enrich these selected representatives with static code context and replay-consistent runtime context, helping determine where an invalid state originates and how it propagates to the visible crash site. Finally, *LLM analysis* completes the semantic diagnosis by combining each representative crash's normalized crash report, triage label, and reconstructed context to determine its type, cause, location, triggering condition, and severity.

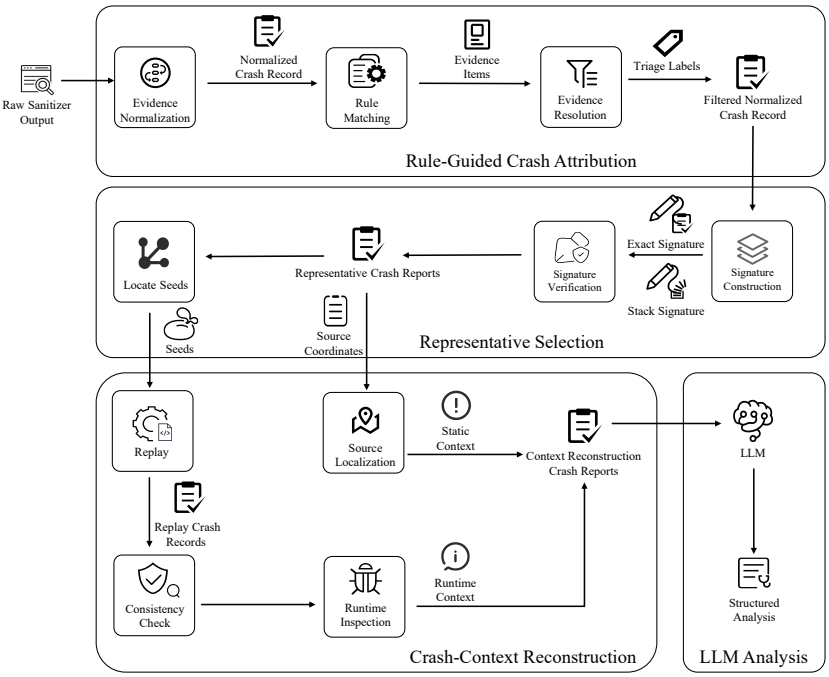


Fig. 2 Multi-stage crash-adjudication pipeline combining rule-guided crash attribution, representative selection, crash-context reconstruction, and LLM analysis.

4.2 Rule-Guided Crash Attribution

Rule-guided crash attribution is the first stage of CrashSage and addresses the false-alert judgment challenge. It consists of sanitizer evidence normalization and rule-based preliminary screening. The former converts raw sanitizer output into a normalized crash report, while the latter assigns a triage label that determines whether the crash is filtered.

4.2.1 Sanitizer Evidence Normalization

Crash reports contain substantial redundant information that is not directly useful for subsequent crash adjudication. CrashSage normalizes each raw crash report into a normalized crash report that retains the information required by subsequent stages. Its fields fall into the six categories summarized below and in Table 1.

Table 1 Key field groups in the normalized crash report

Field Group	Representative Extracted Fields
Failure identity	`sanitizer_type`, `bug_type`, `summary`
Crash-site metadata	`crash_address`, `pc`, `access_type`
Severity and dedup cues	`scariness_score`, `scariness_desc`, `dedup_token`
Structured stack trace	ordered `frames` with function, file, line, and address
Project anchor	`first_in_project_frame`, `first_in_project_function`
Reproduction and status hints	`artifact_path_hint`, `base_unit_hash`, special-condition flags

Table 2 Stack-frame role classification used by CrashSage

Role	Meaning
project	Stack frames in the target-component source code.
driver	Stack frames related to the fuzz driver.
runtime	Runtime-library frames, including libFuzzer, sanitizers, LLVM runtime, libc, etc.
unclassified	Frames that cannot be classified reliably but are not obvious runtime frames.

- **Failure identity** describes the crash report type and its failure category.
- **Crash-site metadata** records the immediate low-level crash information, such as the failing address or access behavior, when it is available.
- **Severity and dedup cues** provide sanitizer- or fuzzer-generated hints that help prioritize and compare crash reports.
- **Structured stack trace** converts the raw call stack into ordered frames for later role modeling.
- **Project anchor** identifies the first visible connection between the crash trace and the target-component source code.
- **Reproduction and status hints** preserve crash-input clues and special execution states that may affect filtering or replay.

4.2.2 Rule-Based Preliminary Screening

Screening begins by matching the function name, file path, and position of each frame in the `structured stack trace` against predefined project, driver, and runtime patterns. Frames without sufficient matching evidence remain *unclassified*. Table 2 summarizes these roles.

Frame ownership alone is not sufficient for false-alert judgment. A crash trace can involve target-component frames, generated-driver frames, and runtime frames at the same time, while the frame where the sanitizer reports the failure may not be the frame that best explains the origin of the crash. CrashSage therefore applies a set of deterministic rules over the ownership and position of stack frames to infer whether the crash is more consistent with noise, a driver-induced failure, or a target-component crash.

Before listing the rules, we define the following notations. Let $S = \langle f_1, \ldots, f_n \rangle$ denote the structured stack trace, ordered from the top crashing frame to lower caller frames. Let F_P, F_D, and F_N denote the sets of project frames, driver frames, and non-runtime frames, respectively, where

$F_N = F_P \cup F_D \cup F_U$ and F_U is the set of unclassified frames. CrashSage computes the driver-frame ratio

$$R_D = \frac{|F_D|}{\max(|F_N|, 1)}, \tag{1}$$

corresponding to `Driver_Frame_Ratio`. Runtime frames are excluded from this ratio because they mainly reflect sanitizer and libFuzzer control flow. Let h_D denote the top-stack depth inspected for driver evidence, θ_D the driver-frame-ratio threshold, m_P the maximum number of project frames treated as shallow project involvement, and θ_S the sanitizer-severity threshold.

CrashSage sets these parameters through an evidence-boundary design strategy that maps each value to an interpretable stack or sanitizer signal. The parameter h_D defines the near-crash stack region used for driver-evidence inspection, θ_D defines the driver-dominance level in the attribution-relevant non-runtime stack, m_P defines the amount of project-code involvement treated as shallow API-boundary execution, and θ_S defines the sanitizer-severity level used to retain project-frame SEGV/null-dereference reports.

For h_D, CrashSage sets $h_D = 3$ because a three-frame window corresponds to the crash frame, the direct caller, and the caller that usually reflects the immediate driver-side invocation context. This value keeps the check centered on the local failure path where argument construction, API invocation, and first observable faulty use are most directly represented. For θ_D, CrashSage sets $\theta_D = 0.8$, using a four-fifths majority as the driver-dominance criterion over non-runtime frames. This value requires driver evidence to occupy a clearly dominant share of the attribution-relevant stack before assigning `likely_driver_bug`, making the label depend on stack-level concentration rather than an isolated driver frame. For m_P, CrashSage sets $m_P = 2$ because shallow API-boundary execution often includes the target entry function and one adjacent helper, initialization, creation, setup, or parsing frame. This value captures brief project-code contact during driver-constructed calls while distinguishing it from deeper target-component execution paths. For θ_S, CrashSage sets $\theta_S = 10$ as the high-severity sanitizer threshold for SEGV/null-dereference reports. This value retains reports when the sanitizer signal indicates severe invalid memory access and the stack simultaneously provides project-frame evidence for target-component analysis.

These values are fixed before evaluating the crash reports, providing a stable and reproducible configuration for the rule-guided attribution stage.

The remaining rule signals also come from the normalized crash report. Timeout, `ALARM`, out-of-memory, LSan-only, and deadly-signal flags are extracted from sanitizer or libFuzzer status lines, while `SCARINESS` comes from the sanitizer severity line. Table 3 lists the 13 rules and their output labels.

The rationale for each rule and its preliminary output label is described below. These labels express the direction supported by the available evidence rather than a final vulnerability verdict.

- **R1.** A `deadly_signal` marker confirms abnormal termination, but the absence of non-runtime frames leaves no project or driver location for attribution. The report is therefore labeled `noise` because it provides no inspectable evidence for the subsequent analysis.
- **R2.** Timeout and `ALARM` markers indicate that execution exceeded its time limit, without providing sanitizer evidence for a memory-access crash. Such reports are labeled `noise` because they do not provide crash evidence for component attribution.
- **R3.** An out-of-memory marker records resource exhaustion, which may depend on allocation demand or execution limits and does not identify a specific faulty access. The rule therefore assigns `noise` rather than attributing the condition to either the driver or the target component.

Table 3 Rules used for crash attribution

Group	Rule	Deterministic Predicate	Output Label
NF	R1	$F_N = \emptyset \wedge$ `deadly_signal`	`noise`
NF	R2	Timeout or `ALARM` marker appears	`noise`
NF	R3	Out-of-memory marker appears	`noise`
NF	R4	LSan-only report without crashing execution	`noise`
DQ	R5	`use_of_uninitialized_value` with a driver frame in the top-h_D stack frames	`likely_driver_bug`
DQ	R6	$R_D \geq \theta_D$	`likely_driver_bug`
DQ	R7	Project init/option/parse function appears near the driver boundary	`likely_driver_bug`
DQ	R8	`stack_buffer_overflow` with the first non-runtime frame in driver code	`likely_driver_bug`
DQ	R9	$F_N = F_D \wedge F_P = \emptyset$	`likely_driver_bug`
DQ	R10	$\lvert F_P \rvert \leq m_P$ and project function matches init/create patterns	`likely_driver_bug`
TC	R11	Memory-safety crash with at least one project frame	`target_crash`
TC	R12	SEGV/null-dereference with `SCARINESS` $\geq \theta_S$ and project frame evidence	`target_crash`
LE	R13	Assertion, abort, or `ABRT` originates in project code	`needs_review`

Note. NF: noise filtering; DQ: driver-quality detection; TC: target-component attribution indication; LE: logic-error indication. F_N, F_D, and F_P denote non-runtime, driver, and project frame sets, respectively.

- **R4.** An LSan-only report identifies unreleased allocations at process exit but contains no crashing instruction, signal, or abort. Because the rule-guided stage concerns the origin of crashes, this report is labeled `noise`.
- **R5.** An uninitialized-value report indicates that data was consumed before receiving a valid value. When driver frames occur within the top h_D frames, the first observable use is close to driver-side execution, so the available evidence supports `likely_driver_bug`.
- **R6.** After runtime frames are removed, $R_D \geq \theta_D$ means that driver frames account for the specified dominant share of the attribution-relevant stack. Because the observable crash path is concentrated in the generated harness, the rule assigns `likely_driver_bug`.
- **R7.** Initialization, option-processing, and parsing functions near the driver boundary are commonly reached while the generated driver constructs objects and prepares API arguments. When project involvement remains at this boundary, the evidence favors invalid driver-side setup over target-component-originated execution, leading to `likely_driver_bug`.
- **R8.** For a stack-buffer overflow, the first non-runtime frame identifies the function whose active stack object exceeds its boundary. If this frame belongs to the driver, the overflowing object is localized to driver-side execution, so the rule assigns `likely_driver_bug`.
- **R9.** If every non-runtime frame belongs to the driver and no project frame is present, the trace contains no observable evidence that execution reached target-component code. The crash is therefore labeled `likely_driver_bug`.
- **R10.** At most m_P project frames limited to initialization or creation functions indicate shallow component involvement during API setup rather than an internal target execution path. This

pattern is treated as `likely_driver_bug` because the evidence primarily reflects how the generated driver configured the call.

- **R11.** A sanitizer-reported memory-safety crash with at least one project frame establishes that the erroneous execution reached target-component code. The rule labels it `target_crash` so that it is retained as a target-component-attributed candidate for subsequent stages, rather than making the final attribution decision in the rule-guided stage.
- **R12.** A severe SEGV or null dereference indicates an invalid memory access, while the presence of project frames anchors that access in target-component execution. Their combination supports the preliminary label `target_crash` and retains the crash for localization.
- **R13.** A project-origin assertion, abort, or `ABRT` identifies the component location that terminates execution, but the report alone cannot distinguish a driver violation of an API precondition from a violated internal invariant. The rule therefore assigns `needs_review` rather than attributing the crash to either side.

Because the rules are evaluated independently, CrashSage resolves their evidence items in the order `noise`, `likely_driver_bug`, `target_crash`, and `needs_review`. The first matched label in this order becomes the triage label; if no rule matches, the crash is assigned `needs_review` by default. This procedure removes reports without usable crash evidence and generated-driver crashes before retaining target-component-attributed candidates and ambiguous cases for further analysis.

Crashes labeled `noise` or `likely_driver_bug` are filtered, whereas those labeled `likely_api_bug` or `needs_review` are kept as retained normalized crash reports for representative selection. For a crash c, let $\mathcal{N}(c)$ denote its normalized crash report and $y(c)$ denote its triage label. The output of rule-guided crash attribution is the pair $\langle \mathcal{N}(c), y(c) \rangle$, which serves as the input to the subsequent stage.

4.3 Representative Selection

Large-scale fuzzing often produces many crash instances that differ in inputs or generated drivers but follow the same underlying failure pattern. Representative selection reduces this repeated analysis cost by comparing exact and stack signatures and forwarding only non-duplicate representatives to the later LLM stage.

Let $\mathcal{C} = \{c_1, \ldots, c_n\}$ denote the set of crashes retained after rule-guided crash attribution. For each crash c, let S_c denote its structured stack trace following the definition of S in Section 4.2, and let $d(c)$ denote the sanitizer deduplication token. Let $b(c)$ denote the sanitizer bug type extracted from $\mathcal{N}(c)$, and let $\ell(c)$ denote the crash-site source location derived from the crash-site metadata or resolved frame location when available. Let $\perp$ denote an unavailable value. The sequence $s(c)$ is the ordered non-runtime key-frame subsequence derived from S_c. Runtime frames are excluded from $s(c)$ because they usually describe sanitizer, libFuzzer, or libc control flow rather than the project-driver execution path.

Exact signature. The exact signature targets crashes with the same sanitizer-provided identity or the same resolved crash site. CrashSage computes

$$\sigma_{\text{exact}}(c) = \begin{cases} H(d(c)), & d(c) \neq \perp, \\ H\big(b(c), \ell(c), \text{top}(s(c))\big), & d(c) = \perp \wedge \ell(c) \neq \perp \wedge |s(c)| > 0, \\ \perp, & \text{otherwise}, \end{cases} \tag{2}$$

where $H(\cdot)$ is a stable hash function and $\mathrm{top}(s(c))$ denotes the first frame in the non-runtime key-frame sequence. The exact-signature match between two crashes is defined as

$$M_{\mathrm{exact}}(c_i, c_j) = \mathbb{I}\big[\sigma_{\mathrm{exact}}(c_i) \neq \bot \wedge \sigma_{\mathrm{exact}}(c_i) = \sigma_{\mathrm{exact}}(c_j)\big]. \tag{3}$$

Here, $M_{\mathrm{exact}}(c_i, c_j) = 1$ represents the strongest duplicate match. The condition holds either because both crashes expose the same sanitizer deduplication token $d(c)$, or because the normalized fields $b(c)$, $\ell(c)$, and $\mathrm{top}(s(c))$ agree. In the latter case, the two reports have the same sanitizer crash class, the same localized crash-site anchor such as file, line, and function, and the same first project/driver frame after runtime frames are removed.

Stack signature. The stack signature targets crashes whose crash-site anchors differ but whose project-driver call paths are identical. Let $p_k(c)$ denote the ordered sequence obtained by taking the first $\min(k, |s(c)|)$ frames from $s(c)$, preserving the original stack order from the top crashing frame to lower caller frames. Here, k is a fixed prefix length used consistently for all crashes. CrashSage builds a stack signature from this bounded prefix:

$$\sigma_{\mathrm{stack}}(c) = H\big(b(c), p_k(c)\big). \tag{4}$$

The bug type is kept in the signature to avoid treating different sanitizer crash classes that happen to pass through similar functions as duplicates. The bounded prefix of the non-runtime stack captures the driver-to-project execution path that reaches the crash. The stack-signature match is defined as

$$M_{\mathrm{stack}}(c_i, c_j) = \mathbb{I}\big[\sigma_{\mathrm{stack}}(c_i) = \sigma_{\mathrm{stack}}(c_j)\big]. \tag{5}$$

Here, $M_{\mathrm{stack}}(c_i, c_j) = 1$ represents a duplicate match based on the retained call path rather than on an identical crash-site anchor. The equality of σ_{stack} requires the same $b(c)$ and the same bounded prefix $p_k(c)$, where $p_k(c)$ is built from the ordered non-runtime frames in $s(c)$. Thus, the two crashes are treated as duplicates when their normalized stack evidence shows the same sanitizer crash class and the same leading project/driver frame sequence, even if the reported line number, resolved crash location, or exact sanitizer token is unavailable or slightly different.

CrashSage evaluates the two match predicates in the order of exact and stack matching, which corresponds to signature verification in Figure 2. For a pair of crashes, stack matching is evaluated only when exact matching does not produce a match. The resulting duplicate decision is

$$M(c_i, c_j) = M_{\mathrm{exact}}(c_i, c_j) \vee M_{\mathrm{stack}}(c_i, c_j). \tag{6}$$

CrashSage processes the retained crashes in discovery order and compares each crash with the representatives selected so far. A matched crash is linked to the corresponding representative, whereas an unmatched crash becomes a new representative. Algorithm 1 summarizes this procedure.

The primary output of representative selection is the selected crash set $\mathcal{R}$. The accompanying mapping $\mathcal{L}$ records each duplicate–representative pair (c_i, r), indicating that c_i matches and reuses the analysis result of representative r.

4.4 Crash-Context Reconstruction

After representative selection, the selected crash representatives still require relevant localization evidence to refine the attribution. A component frame identifies where a failure becomes

Algorithm 1 Representative Selection

Require: retained crashes $\mathcal{C} = \langle c_1, \ldots, c_n \rangle$ in discovery order
Ensure: selected crash set $\mathcal{R}$ and duplicate mapping $\mathcal{L}$

```
R ← ∅, L ← ∅
for c_i ∈ C do
    r* ← ⊥
    for r ∈ R do
        if M(c_i, r) = 1 then
            r* ← r
            break
        end if
    end for
    if r* ≠ ⊥ then
        L ← L ∪ {(c_i, r*)}
    else
        R ← R ∪ {c_i}
    end if
end for
return (R, L)
```

visible, but it does not establish whether the faulty state originates in the generated driver or the target component. Crash-context reconstruction addresses this gap through static code context reconstruction and runtime context reconstruction. Static code context reconstruction maps the driver-to-component call path to source code, while runtime context reconstruction recovers the execution state needed to explain how the failure propagates along that path. Together, they turn an isolated sanitizer crash label into attribution-oriented execution evidence for LLM adjudication of the selected representatives.

4.4.1 Static Code Context Reconstruction

Static code context reconstruction localizes selected project and driver frames in a representative crash report to source files and extracts the surrounding source-code context. CrashSage first maps each selected frame to a local file, function, and line number using the reported source coordinates. Based on these locations, it extracts bounded code windows around the crash-site frame and relevant call-chain frames, preserving the local statements before and after each key frame. These code windows provide source-level evidence for inspecting the crashing operation and tracing how the fuzz input moves from the generated driver into the target API and toward the crash site. However, static code context alone cannot determine the concrete values and object states present in the failing execution.

4.4.2 Runtime Context Reconstruction

Runtime context reconstruction complements static code context reconstruction by recovering execution-state evidence that static code alone cannot provide, including variable values, pointer states, and memory-related conditions at the crash point. This evidence helps determine whether an invalid state is constructed by the driver and propagated into component code or produced within the

Table 4 Evidence groups produced by crash-context reconstruction

Context Type	Evidence Group	Representative Fields	Concrete Role
Static code context	Crash-site anchor	`crash_file`, `crash_line`, `crash_function`, `crash_code_snippet`	Anchors the visible crash site without treating it as the fault origin.
	Call-chain source context	`call_chain_context`, `frame_type`, `code_snippet`	Preserves bounded project/driver frame context for attribution.
Runtime context	Seed recovery and isolation metadata	`seed_locate`, `seed_copy`, `seed_paths.seed`	Records how the triggering artifact is recovered and staged.
	Replay consistency evidence	`repro.runs`, `signature`, `same_signature`, `repro_rate`	Audits whether replay matches the representative crash.
	Runtime execution context	`reproduced`, `backtrace_text`, `registers`, `crash_frame`, `call_stack`	Supplies runtime state for tracing fault propagation and refining attribution.

component itself. CrashSage uses `artifact_path_hint` or `base_unit_hash` in the normalized crash report to locate the triggering seed and stage an isolated copy for replay.

Replaying the seed is necessary to observe the execution state at the crash point. However, environmental differences, timing-dependent behavior, uninitialized state, or external resources may cause the same seed to produce a different crash. CrashSage therefore parses the replayed crash report into a replay-produced normalized crash record and verifies its consistency before collecting runtime context. To bind the replay result to the representative crash, CrashSage builds a replay key

$$\kappa(c) = \langle b(c), \ell(c), s(c) \rangle, \tag{7}$$

where $b(c)$ corresponds to the `bug_type` field of crash report c, $\ell(c)$ denotes the crash-site source location derived from the crash-site metadata or resolved frame location when available, and $s(c)$ denotes the ordered non-runtime key-frame sequence derived from the structured stack trace S. Let c_0 denote the original normalized crash report and c_r denote the replay-produced crash record. CrashSage regards their replay keys as compatible, denoted by $\kappa(c_r) \simeq \kappa(c_0)$, when

$$b(c_r) = b(c_0) \wedge s(c_r) = s(c_0) \wedge \big(\ell(c_0) = \bot \vee \ell(c_r) = \ell(c_0)\big), \tag{8}$$

where $\bot$ denotes an unavailable value in the corresponding field. Only a replay satisfying this condition is treated as consistent with the representative crash and used for runtime inspection.

For a replay-consistent crash, CrashSage invokes GDB as the runtime inspection tool. The bounded GDB run records the reproduced backtrace and crash frame, together with selected registers, function arguments, and local variables from the frames nearest to the crash site. These fields can help reveal state that static code context cannot provide, such as whether a pointer is null, whether a size variable is inconsistent with a buffer, or whether an object has an invalid state before the crashing operation.

Algorithm 2 summarizes this reproduction-gated runtime context reconstruction procedure.

Algorithm 2 Runtime Context Reconstruction

Require: normalized crash report c_0, static code context C_{static}, fuzz target binary B, corpus or artifact directory D
Ensure: reconstructed context $C_{\text{recon}}(c_0) = (C_{\text{static}}, C_{\text{runtime}}, \rho)$

```
u ← LocateSeed(c_0.artifact_path_hint, c_0.base_unit_hash, D)
if u is unavailable then
    return (C_static, ⊥, SeedUnavailable)
end if
u_s ← StageSeed(u)
c_r ← ReplayAndParse(u_s, B)
if c_r is unavailable then
    return (C_static, ⊥, ReplayFailed)
end if
if κ(c_r) ≇ κ(c_0) then
    return (C_static, ⊥, CrashMismatch)
end if
C_runtime ← CollectDebugContext(u_s, B)
if C_runtime is unavailable then
    return (C_static, ⊥, RuntimeContextUnavailable)
end if
ρ ← RuntimeContextAvailable
return (C_static, C_runtime, ρ)
```

LocateSeed, StageSeed, ReplayAndParse, and CollectDebugContext correspond to seed recovery, isolated replay preparation, replay-output parsing, and GDB-based bounded runtime-state collection, respectively. For each representative crash $c \in \mathcal{R}$, the output $C_{\text{recon}}(c)$ contains static code context, optional runtime context, and a status ρ indicating the outcome of seed recovery, replay matching, and GDB collection. If runtime evidence is unavailable, $C_{\text{runtime}} = \perp$, and the static context remains available to subsequent stages. Table 4 organizes the evidence groups by context type and summarizes their representative fields and roles in subsequent analysis.

For each representative crash $c \in \mathcal{R}$, crash-context reconstruction outputs $C_{\text{recon}}(c)$. After this stage, the evidence package consumed by the final LLM analysis is defined as

$$\mathbf{x}(c) = \langle \mathcal{N}(c),\, y(c),\, C_{\text{recon}}(c) \rangle\,. \tag{9}$$

This package combines the normalized crash report and triage label from rule-guided attribution with the reconstructed context produced in this stage.

4.5 LLM Analysis

Given the evidence package $\mathbf{x}(c)$, CrashSage invokes the LLM only for representative crashes $c \in \mathcal{R}$. The LLM produces a structured diagnosis through schema-guided, evidence-guided prompting rather than direct classification over raw sanitizer text. Table 5 summarizes the prompt design sent to the LLM for each representative crash.

Table 5 Prompt design for LLM analysis

Prompt Part	Prompt Design
System role	Act as a crash-adjudication assistant and distinguish the visible crash site from the origin of the faulty state.
Attribution rule	Do not assign target-component attribution unless target-component evidence is stronger than the driver-misuse hypothesis.
Input evidence	Use the triage summary, matched rules, frame-role evidence, normalized sanitizer fields, static context, runtime context, and duplicate-cluster metadata.
Label-conditioned task	For `target_crash`, verify the target-component root-cause hypothesis. For `needs_review`, compare driver-defect and target-component-defect hypotheses.
Examples	No few-shot examples are used.
Output schema	Return `target_attribution`, `root_cause_type`, `root_cause_location`, `triggering_condition`, `severity`, `fix_suggestion`, and `evidence_rationale`.
Decoding and retry	Use `temperature=0`; retry once with a schema-repair instruction if the response is unparsable or incomplete.

For each representative, CrashSage serializes $\mathbf{x}(c)$ into this staged prompt. Runtime evidence is injected when replay succeeds; otherwise, the prompt records the unavailable runtime status instead of hiding the missing evidence.

The analysis task is conditioned on the triage label. For `target_crash`, the prompt asks the model to verify whether the target-component root-cause hypothesis is supported by the reconstructed evidence. For `needs_review`, the prompt asks the model to compare a driver-defect hypothesis against a target-component-defect hypothesis and to identify which side is better supported. This differential framing keeps driver misuse in scope and reduces the tendency to over-attribute a component-visible crash to the target component.

The model output is required to follow a structured schema with the fields `target_attribution`, `root_cause_type`, `root_cause_location`, `triggering_condition`, `severity`, `fix_suggestion`, and `evidence_rationale`. These fields correspond to the following diagnosis tuple:

$$\mathbf{z}(c) = \langle a(c),\, \tau(c),\, \lambda(c),\, \gamma(c),\, \nu(c),\, \phi(c) \rangle\,, \tag{10}$$

where $a(c)$ denotes the final target-component attribution judgment, $\tau(c)$ denotes the root-cause type, $\lambda(c)$ denotes the root-cause location, $\gamma(c)$ denotes the triggering condition, $\nu(c)$ denotes the severity, and $\phi(c)$ denotes the repair suggestion. The `evidence_rationale` field records the short evidence chain supporting these schema fields and remains part of the report metadata. If the second schema-repair attempt still fails validation, the deterministic evidence package is preserved and the LLM diagnosis is marked incomplete. The final structured report is defined as

$$\mathbf{r}(c) = \langle \mathbf{x}(c),\, \mathbf{z}(c) \rangle\,. \tag{11}$$

This report preserves the deterministic evidence prepared by CrashSage together with the LLM-generated semantic diagnosis.

5 Evaluation

To evaluate the performance of CrashSage, we defined three research questions:

- **RQ1: False-alert judgment**: How effectively does rule-guided crash attribution distinguish driver-induced crashes from target-component-attributed crashes?
- **RQ2: Efficiency**: How much redundant analysis does representative selection remove, and what latency do the major pipeline operations introduce?
- **RQ3: Crash localization**: How does crash-context reconstruction improve crash attribution and root-cause localization?

5.1 Experimental Setup

CrashSage is implemented in Python and uses Qwen3-Coder-Plus, accessed through a cloud API, as the LLM backend. All experiments are conducted on a machine running Ubuntu 22.04 and equipped with an AMD Ryzen 5 5600G with Radeon Graphics processor.

We evaluate CrashSage on 12 third-party open-source libraries used in connected-vehicle software contexts. The selected libraries include c-ares, cJSON, expat, faad2, jansson, json-c, libmad, libmspack, libpcap, libsndfile, tinyxml2, and zlib, covering connected-vehicle software functions such as networking, structured-data parsing, packet capture, compression, media decoding, and archive processing. Table 6 summarizes their functions, API scales, and representative deployment roles.

Table 6 Third-party open-source libraries used in the evaluation

Library	Function	APIs	Crash Reports	Deployment Role
c-ares	Asynchronous DNS resolution	27	103	Gateway DNS
cJSON	JSON parsing and generation	60	78	Config/telemetry parsing
expat	XML parsing	12	36	Configuration parsing
faad2	AAC audio decoding	12	151	Media decoding
jansson	JSON parsing and generation	12	128	Config/telemetry parsing
json-c	JSON parsing and generation	12	40	Config/telemetry parsing
libmad	MPEG audio decoding	12	112	Media decoding
libmspack	Archive extraction	12	54	Package/archive processing
libpcap	Packet capture and filtering	30	115	Network monitoring
libsndfile	Audio file processing	12	76	Media/log processing
tinyxml2	XML parsing	13	63	Configuration parsing
zlib	Compression and decompression	29	65	OTA/log compression
Combined	–	**243**	**1021**	–

We collect crash reports following the fuzz-driver generation and fuzzing workflow of CKGFuzzer (Xu et al. 2025). The generated drivers are compiled and executed by libFuzzer with ASan+UBSan instrumentation in Docker containers based on the OSS-Fuzz base-runner image. The testing process produces 1021 crash reports in total.

To evaluate the contributions of rule-guided crash attribution, representative selection, and crash-context reconstruction, we construct four controlled variants:

- **V0** uses CKGFuzzer's original crash-cause classification module, which performs prompt-based classification using the raw crash report, fuzz-driver source, target-component source, and LLM-extracted error patterns.
- **V1** augments V0 with rule-guided attribution labels and filtering decisions.
- **V2** combines rule-guided crash attribution and representative selection without crash-context reconstruction.
- **V3 (CrashSage)** integrates rule-guided crash attribution, representative selection, and crash-context reconstruction into the complete pipeline.

Representative selection and crash-context reconstruction are evaluated after rule-guided crash attribution. Representative selection determines which retained reports are submitted to LLM analysis, and representative decisions are propagated to duplicate reports for per-crash scoring. Crash-context reconstruction supplies additional evidence for the selected representatives. We therefore do not construct variants that apply either module independently without rule-guided crash attribution.

After public-reference validation, local reproduction, and inspection of all crash reports, we label a crash as true only when it is locally reproducible, its corresponding vulnerability can be verified from public sources such as CVE records, official advisories, or public issue/patch records, and its failure is attributable to target-component API behavior rather than generated-driver misuse. Crashes that are not reproducible, lack sufficient target-component evidence, or are manually confirmed as driver-induced are labeled false. Precision, recall, and F1 are computed using these ground-truth labels at the original crash-report level. When representative selection is used, each representative diagnosis and ground-truth label are propagated to its duplicate reports before computing TP, FP, and FN.

5.2 Evaluation of Rule-Guided Crash Attribution

To evaluate rule-guided crash attribution, we compare V1 with V0 on the same 1021 crash reports. Reports labeled `likely_api_bug` or `needs_review` are forwarded to subsequent analysis after reports labeled `noise` or `likely_driver_bug` are filtered. Precision measures the proportion of predicted API-bug crashes that are labeled true, recall measures the proportion of true API-bug crashes finally identified by the variant, and F1 reflects their balance.

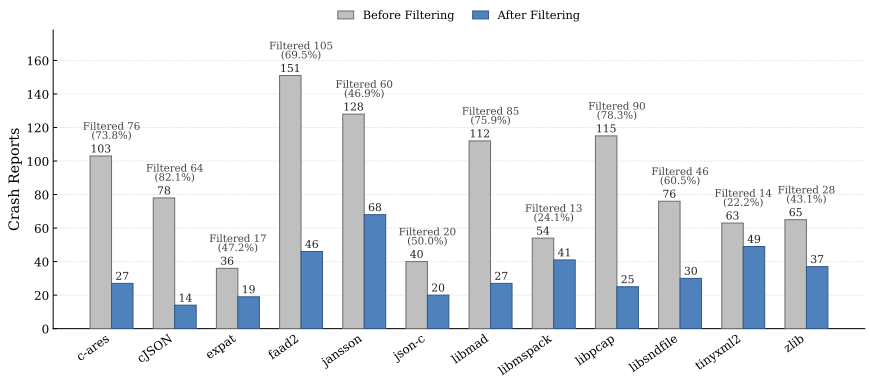


Fig. 3 Reduction in crash reports after rule-guided crash attribution.

Table 7 Ground-truth audit of rule-guided filtering.

Triage Outcome	Ground-truth True	Ground-truth False	Total
`noise`	0	103	103
`likely_driver_bug`	0	515	515
Filtered	**0**	**618**	**618**
Retained	146	257	403
Combined	**146**	**875**	**1021**

Table 8 Effect of rule-guided crash attribution.

Variant	TP	FP	FN	Precision	Recall	F1
V0	125	298	21	0.2955	0.8562	0.4394
V1	135	157	11	0.4623	0.9247	0.6164

The experimental results are shown in Figure 3, Tables 7, 8, and 9. Rule-guided crash attribution filters 618 reports from 1021 crash reports and retains 403 reports for subsequent analysis. Table 7 shows that all filtered reports are labeled false, whereas all 146 reports labeled true are retained. Thus, rule-guided filtering introduces no ground-truth false negatives at the filtering stage. V1 achieves precision 0.4623, recall 0.9247, and F1 0.6164, with 11 false negatives. These false negatives are retained reports that remain misclassified rather than reports removed by the filtering rules.

This result is partly attributable to the reports removed by rule-guided crash attribution. Reports with insufficient crash evidence or evidence more consistent with driver-induced failures can introduce misleading information into subsequent analysis. Filtering these reports reduces driver-side

Table 9 Per-library effect of rule-guided crash attribution.

Library	V0			V1		
	Prec.	**Recall**	**F1**	**Prec.**	**Recall**	**F1**
c-ares	0.1194	1.0000	0.2133	0.3200	1.0000	0.4848
cJSON	0.0000	0.0000	0.0000	0.4615	1.0000	0.6316
expat	0.7143	0.3846	0.5000	0.7500	0.2308	0.3529
faad2	0.2121	1.0000	0.3500	0.9333	1.0000	0.9655
jansson	0.2131	1.0000	0.3514	0.1912	1.0000	0.3210
json-c	0.6000	0.4615	0.5217	0.6667	0.9231	0.7742
libmad	0.3250	1.0000	0.4906	0.4815	1.0000	0.6500
libmspack	0.7222	1.0000	0.8387	0.7222	1.0000	0.8387
libpcap	0.2979	1.0000	0.4590	0.6667	1.0000	0.8000
libsndfile	0.4643	1.0000	0.6341	0.8667	1.0000	0.9286
tinyxml2	0.2500	1.0000	0.4000	0.2653	1.0000	0.4194
zlib	0.6842	1.0000	0.8125	0.6842	1.0000	0.8125

cases entering later LLM analysis while preserving API-bug candidates that warrant further analysis. In response to RQ1, rule-guided crash attribution distinguishes driver-induced crashes from target-component-attributed crashes under the ground-truth labels.

5.3 Evaluation of Representative Selection

To evaluate the efficiency of representative selection, we compare V0 and V3. We first examine the time required by the major CrashSage modules and then distinguish the reduction caused by rule-guided filtering from the reduction caused by representative selection over the retained reports.

Table 10 Latency of the major modules in CrashSage.

Module	Typical Time per Report
Rule-guided crash attribution	< 10 ms
Crash-context reconstruction	˜2,000–15,000 ms
Representative selection	< 5 ms
LLM analysis	˜30,000–60,000 ms

As shown in Table 10, rule-guided crash attribution and representative selection each require less than 10 ms per report. In contrast, crash-context reconstruction requires approximately 2–15 s because it replays the crashing input and collects runtime state, while LLM analysis requires approximately 30–60 s because it depends on remote model inference. These two modules therefore account for most of the analysis time. The cost of crash-context reconstruction is partly reduced by rule-guided crash attribution, which filters 618 of the 1021 crash reports before reconstruction is performed.

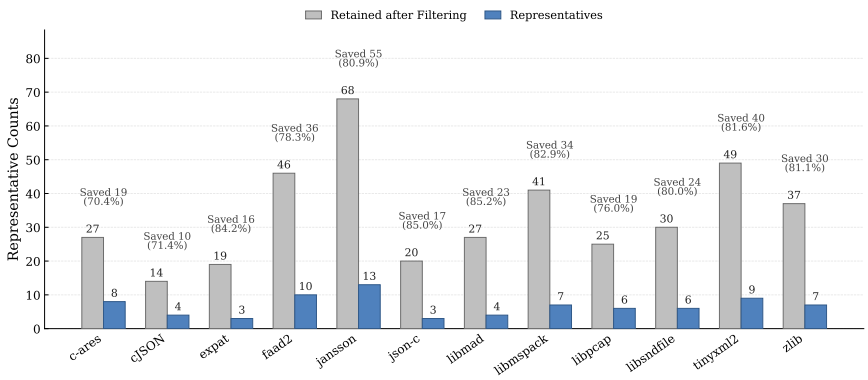


Fig. 4 Reduction in crash reports achieved by representative selection.

Figure 4 presents the reduction achieved by representative selection over the 403 reports retained after rule-guided crash attribution. Overall, 80 reports are submitted as representatives, removing 323 repeated or redundant reports.

These reductions combine two distinct effects. Rule-guided crash attribution first reduces the candidate set from 1021 raw reports to 403 retained reports. Representative selection further keeps 80 representatives. Considering both rule-guided filtering and representative selection, CrashSage reduces end-to-end LLM submissions from 1021 to 80 (92.16%).

The reduction varies across libraries because the generated crashing inputs contain different levels of repetition. RQ2 is reported once because it measures the number of reports submitted to LLM analysis, which is determined by CrashSage before final diagnosis. In response to RQ2, rule-guided filtering first removes low-value or likely driver-induced reports, and representative selection then reduces repeated LLM analysis over the retained candidate set by keeping one report for each group of crashes with matching exact or stack signatures.

5.4 Evaluation of Crash-Context Reconstruction

To evaluate crash-context reconstruction, we compare V2 and V3 on the same crash reports. Both variants use rule-guided crash attribution and representative selection, while only V3 includes crash-context reconstruction. With the ground-truth labels held fixed, this comparison evaluates the effect of adding crash-context reconstruction to V2.

Table 11 Effect of crash-context reconstruction.

Variant	TP	FP	FN	Precision	Recall	F1
V2	135	157	11	0.4623	0.9247	0.6164
V3	146	62	0	0.7019	1.0000	0.8249

The experimental results are shown in Table 11 and Table 12. Compared with V2, V3 achieves precision 0.7019, recall 1.0000, and F1 0.8249 under the ground-truth labels.

Table 12 Per-library effect of crash-context reconstruction.

Library	V2			V3		
	Prec.	**Recall**	**F1**	**Prec.**	**Recall**	**F1**
c-ares	0.3200	1.0000	0.4848	1.0000	1.0000	1.0000
cJSON	0.4615	1.0000	0.6316	1.0000	1.0000	1.0000
expat	0.7500	0.2308	0.3529	0.6842	1.0000	0.8125
faad2	0.9333	1.0000	0.9655	0.9333	1.0000	0.9655
jansson	0.1912	1.0000	0.3210	0.3095	1.0000	0.4727
json-c	0.6667	0.9231	0.7742	0.6500	1.0000	0.7879
libmad	0.4815	1.0000	0.6500	1.0000	1.0000	1.0000
libmspack	0.7222	1.0000	0.8387	0.7222	1.0000	0.8387
libpcap	0.6667	1.0000	0.8000	0.7368	1.0000	0.8485
libsndfile	0.8667	1.0000	0.9286	0.8667	1.0000	0.9286
tinyxml2	0.2653	1.0000	0.4194	0.9286	1.0000	0.9630
zlib	0.6842	1.0000	0.8125	0.6842	1.0000	0.8125

This improvement is associated with the localization evidence added by crash-context reconstruction. A crash may become visible inside a target-component API even when the faulty state originates from the generated driver. Static code context links the visible crash site to the driver-to-component call path, while runtime context records the state observed during execution. In response to RQ3, these static and runtime evidence sources help distinguish the visible crash location from the origin of the faulty state under the ground-truth labels.

5.5 Case Studies

To complement the quantitative results above, we present three case studies that illustrate how CrashSage prepares structured crash evidence before final LLM analysis. They are selected to match the three evaluation concerns studied above: false-alert judgment, efficiency, and crash localization. Table 13 summarizes three case studies corresponding to the three evaluation concerns.

In Case 1, the sanitizer reports a `heap-buffer-overflow`, but the top crashing frame and heap-boundary evidence are both confined to the generated driver. The driver allocates a 20-byte `second_str` buffer and then writes the terminator after copying fuzz input into that buffer, so the faulty state is created before any target-component behavior needs to be considered. CrashSage therefore treats this report as `likely_driver_bug` and filters it before LLM analysis, illustrating how rule-guided attribution prevents a driver memory error from being counted as a target-component crash.

In Case 2, the libpcap reports form a repeated SEGV group rather than independent failures. The raw reports share the same crash-site evidence and stack signature, and CrashSage records reports such as `driver_30_crash_2` and `driver_30_crash_3` as duplicates of the representative `driver_30_crash_1` in a cluster of more than 10 reports. Representative selection therefore skips duplicate LLM submissions and reuses the representative mapping, providing a concrete instance of the RQ2 result: redundant SEGV reports are collapsed by signature evidence before final analysis.

In Case 3, the raw prompt-based analysis does not classify the json-c crash as an API-bug, although the crash is labeled true under the ground-truth labels. CrashSage changes this decision after

Table 13 Case studies for CrashSage evaluation

Evidence	Case 1: False-alert judgment	Case 2: Efficiency	Case 3: Crash localization
Crash report	cJSON `driver_4_crash_1`	libpcap `driver_30` group	json-c `driver_13_crash_1`
Failure identity	`heap-buffer-overflow`	repeated `SEGV`	`SEGV` wild write
Attribution evidence	top crashing frame: driver	shared crash-site / stack signature	project frame in `printbuf_memappend`
Context or mapping	20-byte driver heap region	duplicate–representative pair; 10+ reports	state-sensitive `printbuf` context; replay confirmed
Pipeline output	`likely_driver_bug`; filtered	duplicate skipped; representative reused	context-supported API-bug diagnosis

reconstructing the state-sensitive context around `printbuf_memappend`. The context records that the driver-created `printbuf` reaches the library with boundary-sensitive position and size fields. Replay confirms that this state consistently reaches the same failing library path. The reconstructed data flow shows that the generated driver directly sets `pb->bpos` near `INT_MAX` while keeping `pb->size` small before invoking `printbuf_memappend`. The additional static and runtime context turns a raw-report-level negative judgment into a confirmed API-bug diagnosis under the ground-truth labels.

6 Limitations

The current implementation follows one LLM-assisted fuzz-driver generation workflow and uses ASan+UBSan instrumentation. This setting is consistent across all evaluated variants, which keeps the comparison controlled, but other driver generators, prompt strategies, fuzzing engines, or sanitizer configurations may change the proportion of driver-induced crashes, duplicate reports, and target-component candidates. Extending CrashSage to additional driver-generation workflows and sanitizer combinations is left as future work.

Rule-guided crash attribution is based on a predefined set of deterministic rules over sanitizer signals and stack-frame roles. These rules are designed to conservatively remove reports with insufficient crash evidence or evidence more consistent with driver-induced failures, but their coverage is still bounded by the sanitizer information and stack structure available in each report. Crash-context reconstruction depends on replaying the crashing input and collecting runtime state in a stable execution environment. When replay fails or the reproduced crash does not match the original signature, CrashSage falls back to the normalized crash report and static context, which may reduce localization confidence. Different LLM capabilities or false-alert ratios may change the observed gains. CrashSage is therefore most appropriate as an evidence-preparation and attribution aid: it preserves reports that warrant further analysis and records the evidence used for each filtering or retention decision.

7 Conclusion

CrashSage introduces an evidence-guided crash-adjudication pipeline for LLM-assisted fuzzing workloads. By applying rule-guided crash attribution, representative selection, and crash-context

reconstruction before final LLM analysis, it replaces direct reasoning over raw crash reports with analysis over filtered, representative, and reconstructed crash evidence. In particular, representative selection reduces repeated crash instances before LLM analysis, and crash-context reconstruction combines static code context with replay-consistent runtime context for the selected representatives to distinguish the visible crash site from the origin and propagation of the faulty state. Evaluation on 1021 crash reports collected from 12 third-party open-source libraries shows that CrashSage achieves 70.19% precision, 100.00% recall, and an 82.49% F1 score under ground-truth labels. The full pipeline also reduces end-to-end LLM submissions from 1021 to 80 (92.16%). Rule-guided crash attribution and representative selection each require less than 10 ms per report, whereas crash-context reconstruction requires approximately 2–15 s and LLM analysis requires approximately 30–60 s per case. These results demonstrate that CrashSage improves attribution quality while substantially reducing redundant LLM submissions. Future work will extend the evaluation to additional driver-generation workflows and sanitizer configurations.

References

Bernardini C, Asghar MR, Crispo B (2017) Security and privacy in vehicular communications: Challenges and opportunities. Vehicular Communications 10:13–28. https://doi.org/10.1016/j.vehcom.2017.10.002

Böhme M, Pham VT, Nguyen MD, et al (2019) Directed greybox fuzzing. Communications of the ACM 62(9):90–97. https://doi.org/10.1145/3339252

Checkoway S, et al (2011) Comprehensive experimental analyses of automotive attack surfaces. In: Proceedings of the USENIX Security Symposium, URL https://www.usenix.org/conference/usenix-security-11/comprehensive-experimental-analyses-automotive-attack-surfaces

Dang Y, Wu R, Zhang H, et al (2012) ReBucket: A method for clustering duplicate crash reports based on call stack similarity. In: Proceedings of the International Conference on Software Engineering, pp 1084–1093, https://doi.org/10.1109/ICSE.2012.6227101

Deng Y, et al (2023) Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis, pp 423–435, https://doi.org/10.1145/3597926.3598067

Deng Y, et al (2024) Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 70:1–70:13, https://doi.org/10.1145/3597503.3623343

Ding Z, Le Goues C (2021) An empirical study of OSS-Fuzz bugs. In: Proceedings of the IEEE/ACM International Conference on Mining Software Repositories, pp 131–142, https://doi.org/10.1109/MSR52588.2021.00026

Ghosh R, von Stockhausen HM, Schmitt M, et al (2025) CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 28757–28765, https://doi.org/10.1609/aaai.v39i28.35139

Huang Y, et al (2024) CrashTranslator: Automatically reproducing mobile application crashes directly from stack trace. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, https://doi.org/10.1145/3597503.3623298

Jiang Z, et al (2021) Igor: Crash deduplication through root-cause clustering. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp 3318–3336, https://doi.org/10.1145/3460120.3485364

Joshy AK, Le W (2022) FuzzerAid: Grouping fuzzed crashes based on fault signatures. In: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, pp 44:1–44:12, https://doi.org/10.1145/3551349.3556959

Ladisa P, Plate H, Martinez M, et al (2023) SoK: Taxonomy of attacks on open-source software supply chains. In: Proceedings of the IEEE Symposium on Security and Privacy, pp 1509–1526, https://doi.org/10.1109/SP46215.2023.10179304

Lemieux C, Sen K (2018) FairFuzz: Targeting rare branches to rapidly increase greybox fuzz testing coverage. In: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, pp 475–485, https://doi.org/10.1145/3238147.3238176

Lyu Y, et al (2024) Prompt Fuzzing for fuzz driver generation. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp 3793–3807, https://doi.org/10.1145/3658644.3670396

Meng R, et al (2024) Large language model guided Protocol Fuzzing. In: Proceedings of the Network and Distributed System Security Symposium, URL https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing/

Petit J, Shladover SE (2015) Potential cyberattacks on automated vehicles. IEEE Transactions on Intelligent Transportation Systems 16(2):546–556. https://doi.org/10.1109/TITS.2014.2342271

Rodrigues IM, et al (2022) FaST: A linear time stack trace alignment heuristic for crash report deduplication. In: Proceedings of the IEEE/ACM International Conference on Mining Software Repositories, pp 270–281, https://doi.org/10.1145/3524842.3527951

Sanvito D, Arriciati G, Siracusano G, et al (2025) AutoCVSS: Assessing the performance of LLMs for automated software vulnerability scoring. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing: Industry Track, pp 564–575, https://doi.org/10.18653/v1/2025.emnlp-industry.38

Serebryany K, et al (2012) AddressSanitizer: A fast address sanity checker. In: Proceedings of the USENIX Annual Technical Conference, URL https://www.usenix.org/system/files/conference/atc12/atc12-final39.pdf

Shetty M, et al (2022) DeepAnalyze: Learning to localize crashes at scale. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 549–560, https://doi.org/10.1145/3510003.3512759

Stepanov E, Serebryany K (2015) MemorySanitizer: Fast detector of uninitialized memory use in C++. In: Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization, https://doi.org/10.1109/CGO.2015.7054186

Sun Q, et al (2022) A survey on cyber-security of connected and autonomous vehicles (CAVs). IEEE Transactions on Intelligent Transportation Systems 23(7):6240–6259. https://doi.org/10.1109/TITS.2021.3085297

Wang J, Huang Y, Chen C, et al (2024) Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering 50(4):911–936. https://doi.org/10.1109/TSE.2024.3368208

Wu R, Zhang H, Cheung SC, et al (2014) CrashLocator: Locating crashing faults based on crash stacks. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis, pp 204–214, https://doi.org/10.1145/2610384.2610386

Xia CS, et al (2024) Fuzz4All: Universal fuzzing with large language models. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 1547–1559, https://doi.org/10.1145/3597503.3639121

Xu H, et al (2025) CKGFuzzer: LLM-based fuzz driver generation enhanced by code knowledge graph. In: Proceedings of the IEEE/ACM International Conference on Software Engineering: Companion Proceedings, pp 243–254, https://doi.org/10.1109/ICSE-COMPANION66252.2025.00079

Yan J, et al (2024) Better debugging: Combining static analysis and LLMs for explainable crashing fault localization. CoRR abs/2408.12070. https://doi.org/10.48550/arXiv.2408.12070, arXiv:2408.12070